

shark on wire 2

 **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

We found this packet capture. Recover the flag.

Tools Used

- `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read PCAP metadata
 - **Wireshark** – GUI tool for analyzing network traffic
 - `tshark` – Command-line Wireshark tool for packet analysis
 - `awk` – Text processing and field extraction
 - `sed` – Stream editor for text manipulation
 - `tr` – Translate or delete characters
 - `python` – Convert ASCII codes to characters
-

Complete Solution

Step 1: Download and Prepare

Download the PCAP file named `capture.pcap` from the challenge page.

Step 2: Basic File Inspection

First, verify the file type and general information:

```
file capture.pcap
```

Expected output:

```
capture.pcap: pcap capture file, microsecond ts (little-endian) - version 2.4
(Ethernet, capture length 262144)
```

The file is a **PCAP** capture file containing network traffic.

Step 3: Search for Visible Text

Check if the flag appears as plain text:

```
strings capture.pcap | grep -i "picoCTF"
```

Output:

```
picoCTF Sure is fun!CmP]2
I really want to find some picoCTF flagsEmP]m=
picoCTF Sure is fun!ImP]
I really want to find some picoCTF flagsKmP]
picoCTF Sure is fun!OmP]
I really want to find some picoCTF flagsQmP]
picoCTF Sure is fun!VmP]\V
I really want to find some picoCTF flagsXmP]
picoCTF Sure is fun!\mP]
I really want to find some picoCTF flags^mP]
picoCTF Sure is fun!bmP]I
I really want to find some picoCTF flagsdmP]
picoCTF Sure is fun!hmP]
I really want to find some picoCTF flagskmP]
picoCTF Sure is fun!omP]
I really want to find some picoCTF flagsqmP]}}
picoCTF Sure is fun!umP]
I really want to find some picoCTF flagswmP]
```

The flag isn't directly visible as a continuous string. We need to analyze the network traffic further.

Step 4: Examine Metadata

```
exiftool capture.pcap
```

Output:

```
ExifTool Version Number      : 13.50
File Name                    : capture.pcap
Directory                    : .
File Size                    : 112 kB
File Modification Date/Time   : 2026:05:17 10:35:46+02:00
File Access Date/Time        : 2026:05:18 17:15:26+02:00
File Inode Change Date/Time   : 2026:05:17 10:35:55+02:00
File Permissions              : -rw-rw-rw-
File Type                    : PCAP
File Type Extension          : pcap
MIME Type                    : application/vnd.tcpdump.pcap
PCAP Version                 : PCAP 2.4
Byte Order                   : Little-endian (Intel, II)
Link Type                    : BSD Loopback
Time Stamp                   : 2019:08:11 21:31:18.146248+02:00
```

The metadata doesn't reveal the flag.

Step 5: Analyze with Wireshark

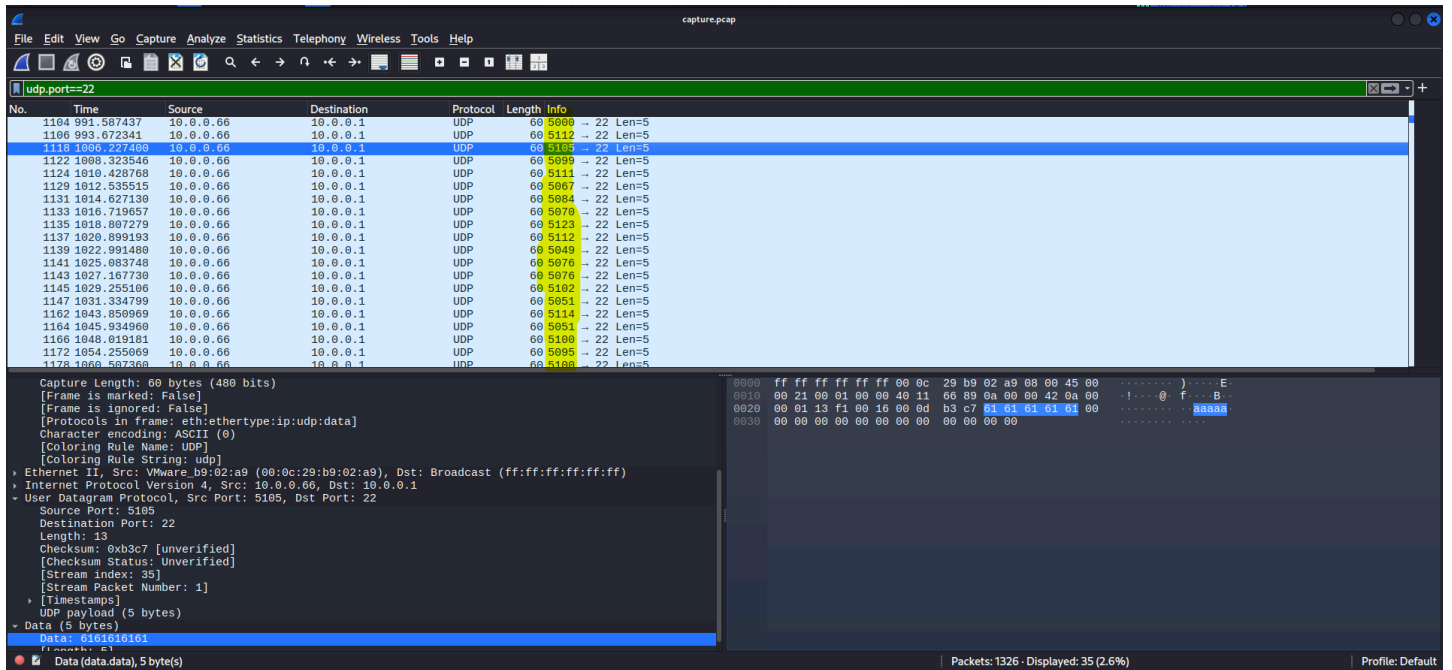
Open the file in Wireshark to get a visual understanding of the traffic:

Wireshark packet capture showing various network traffic. The packet list on the left shows a mix of protocols including MDNS, TCP, ARP, and UDP. The packet details pane on the right shows the structure of a selected packet, including Ethernet II, Internet Protocol Version 6, and User Datagram Protocol fields. The packet bytes pane on the right shows the raw hex and ASCII data of the selected packet.

At first glance, nothing seems unusual. However, when we look closer at **UDP port 22** traffic, we notice something interesting – there’s a “start” pattern, but the flag is hidden.

Wireshark packet capture filtered for UDP port 22. The packet list on the left shows a series of UDP packets from source 10.0.0.66 to destination 10.0.0.1. The packet details pane on the right shows the structure of a selected packet, including Ethernet II, Internet Protocol Version 4, and User Datagram Protocol fields. The packet bytes pane on the right shows the raw hex and ASCII data of the selected packet, highlighting a "Start" pattern in the ASCII data.

Upon careful observation of the traffic, the **Source Port** values vary significantly between packets.



Key finding: UDP packets on port 22 appear to have source ports that might encode data.

Step 6: Filter UDP Port 22 Traffic

Let's filter all UDP packets on port 22:

```
tshark -nr capture.pcap -Y "udp.port==22"
```

Output (excerpt):

1104	991.587437	10.0.0.66 → 10.0.0.1	UDP 60 5000 → 22 Len=5
1106	993.672341	10.0.0.66 → 10.0.0.1	UDP 60 5112 → 22 Len=5
1118	1006.227400	10.0.0.66 → 10.0.0.1	UDP 60 5105 → 22 Len=5
1122	1008.323546	10.0.0.66 → 10.0.0.1	UDP 60 5099 → 22 Len=5
1124	1010.428768	10.0.0.66 → 10.0.0.1	UDP 60 5111 → 22 Len=5
1129	1012.535515	10.0.0.66 → 10.0.0.1	UDP 60 5067 → 22 Len=5
1131	1014.627130	10.0.0.66 → 10.0.0.1	UDP 60 5084 → 22 Len=5
1133	1016.719657	10.0.0.66 → 10.0.0.1	UDP 60 5070 → 22 Len=5
1135	1018.807279	10.0.0.66 → 10.0.0.1	UDP 60 5123 → 22 Len=5
1137	1020.899193	10.0.0.66 → 10.0.0.1	UDP 60 5112 → 22 Len=5
1139	1022.991480	10.0.0.66 → 10.0.0.1	UDP 60 5049 → 22 Len=5
1141	1025.083748	10.0.0.66 → 10.0.0.1	UDP 60 5076 → 22 Len=5
1143	1027.167730	10.0.0.66 → 10.0.0.1	UDP 60 5076 → 22 Len=5
1145	1029.255106	10.0.0.66 → 10.0.0.1	UDP 60 5102 → 22 Len=5
1147	1031.334799	10.0.0.66 → 10.0.0.1	UDP 60 5051 → 22 Len=5

...

1303 1171.059146 10.0.0.80 → 10.0.0.1 UDP 60 5000 → 22 Len=3

The **source ports** (the 5th field in the output) are all 4-digit numbers starting with 5. This is suspicious!

Step 7: Extract the Source Ports

Extract just the source port numbers (field 8 in the tshark output):

```
tshark -nr capture.pcap -Y "udp.port==22" | awk '{print $8}'
```

Output:

```
5000
5112
5105
5099
5111
5067
5084
5070
5123
5112
5049
5076
5076
5102
5051
5114
5051
5100
5095
5100
5097
5116
5097
5095
5118
5049
5097
5095
```

5115
5116
5051
5103
5048
5125
5000

Step 8: Remove the Leading Digits

Notice that all source ports start with `5`. Let's remove the first digit (`5`) and then any leading zeros:

```
tshark -nr capture.pcap -Y "udp.port==22" | awk '{print $8}' | sed 's/^5//g' | sed 's/^0//g' | tr '\n' ' '
```

Output:

```
00 112 105 99 111 67 84 70 123 112 49 76 76 102 51 114 51 100 95 100 97 116 97 95 118  
49 97 95 115 116 51 103 48 125 00
```

We get a sequence of numbers! The numbers `112 105 99 111 67 84 70 123 ...` look like **ASCII codes**.

Step 9: Convert ASCII Codes to Characters

The sequence `112, 105, 99, 111, 67, 84, 70, 123, ...` corresponds to ASCII characters. Let's decode it:

Using Python:

```
codes = [112, 105, 99, 111, 67, 84, 70, 123, 112, 49, 76, 76, 102, 51, 114, 51, 100, 95, 100, 97, 116, 97, 95, 118, 49, 97, 95, 115, 116, 51, 103, 48, 125]  
result = ''.join(chr(code) for code in codes)  
print(result)
```

Output:

Step 10: Alternative – Decode with CyberChef

You can also decode the ASCII codes using [CyberChef](#):

1. Go to <https://gchq.github.io/CyberChef>
 2. Paste the numbers: 112 105 99 111 67 84 70 123 ...
 3. Add the "From Decimal" operation
 4. The decoded flag appears instantly
-

One-Liner Solution

Here's the complete pipeline to extract and decode the flag in one command:

```
tshark -nr capture.pcap -Y "udp.port==22" | awk '{print $8}' | sed 's/^5//g' | sed 's/^0//g' | tr '\n' ' ' | python3 -c "import sys; print(''.join(chr(int(x)) for x in sys.stdin.read().split() if x))"
```

Final Result

picoCTF{<REDACTED>}

Note: The actual flag is redacted here. In the real challenge, running the commands above will reveal the complete flag.

Key Concepts

Concept	Description
UDP Source Ports	Source port numbers can be manipulated to encode hidden data.
ASCII Codes	Each character has a corresponding decimal ASCII value (e.g., <code>112</code> = 'p').
tshark	Command-line tool for extracting specific fields from PCAP files.
awk	Text processing tool for extracting specific columns from output.
sed	Stream editor for removing leading characters (<code>s/^5//g</code> removes the first digit).
ASCII Decoding	Converting decimal ASCII codes back to characters reveals the hidden message.

Pro Tips

1. **Look for unusual port numbers** – Source ports in the 5000-5125 range are uncommon. This is a red flag that data might be encoded in them.
2. **Check UDP traffic** – Flags aren't always in TCP or HTTP. UDP packets can also carry hidden data.
3. **Remove leading digits** – If all numbers share a common prefix (like `5`), removing it reveals smaller values (likely ASCII codes).
4. **ASCII codes are usually 2-3 digits** – Values between 65-122 correspond to letters, 48-57 to digits, and 123/125 to `{` and `}`.
5. **Use sed to clean data** – `sed 's/^5//g'` removes the first character `5`; `sed 's/^0//g'` removes leading zeros.
6. **Pipeline power** – Combining `tshark`, `awk`, `sed`, `tr`, and `python` extracts, cleans, and decodes in one go.

Alternative ASCII Decoding Tools

Tool	Command / Link
Python	<pre>python3 -c "print(''.join(chr(int(x)) for x in '112 105 99...'.split()))"</pre>
CyberChef	https://gchq.github.io/CyberChef/ (Operation: From Decimal)
Online	https://www.asciitohex.com/